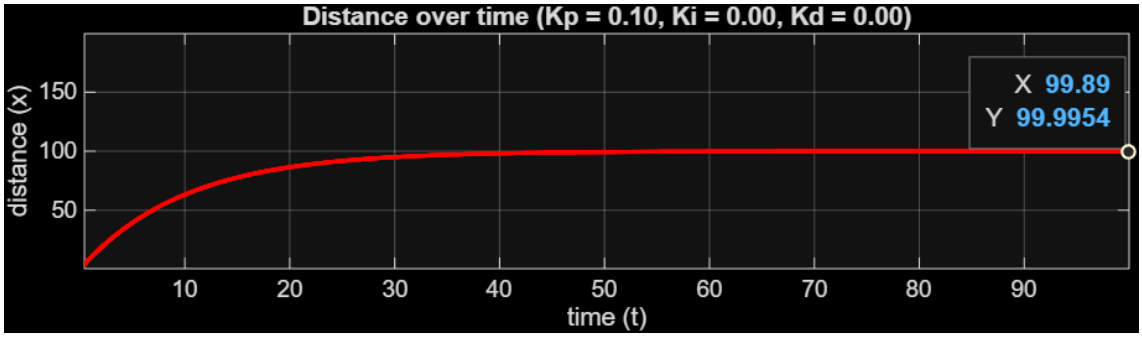
KENZIE AZURA SYAKI 5022251140 – TASK II

The vessels height relative to the watere surface (Heave/**Alittude**) and the vessel's orientation (especially its **pitch and roll**) are important values to control to create a steady cruising. There are a bunch of ways we can configure a control system to give instruction on how to actuate the hydrofoil, and one of them is the PID controller.

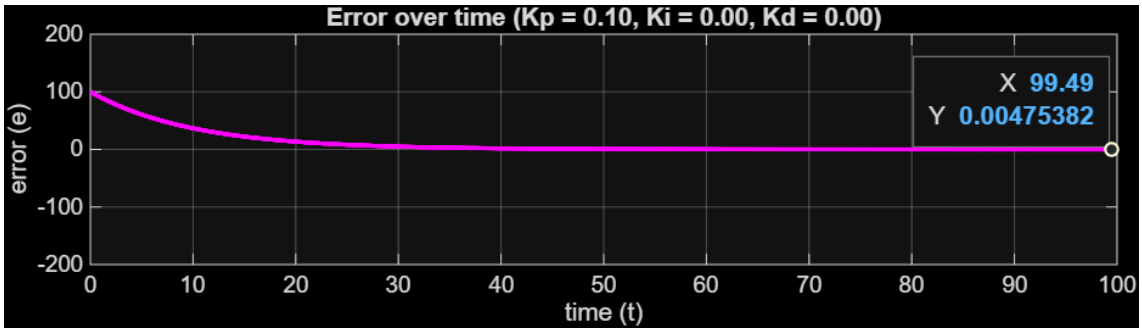
We start from the simplest control loop we can think of: lets say we want our system to move forward 100 meters from its origin (this is what we call a **set point**), and by using sensors, we could feed information on how far we are from the set point over time t (or usually called the **error**, with the symbol e(t)). In other words, we calculate the error by the set point-position traveled by the actuator (**plant**) which is the mechanical system that executes the job. This error then feeded **proportionally** to set a certain speed for which the plant will execute. This is what we call the proportional (P) loop. We tune how fast our system moving by multiplying e(t) by a constant **Kp**. For example, when t = 0, e(t) will be the same as our set point 100m, and if we set Kp to be 1, then the initial speed of our system would be 100 times 1 which is 100m/s. The point is, by tuning the right Kp for our system, we will get the most effective linear correlation between the error with the speed.

$$K_p e(t)$$

(a)

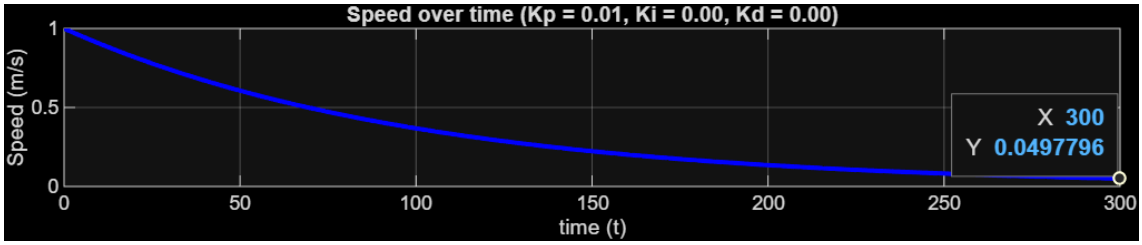


(b)

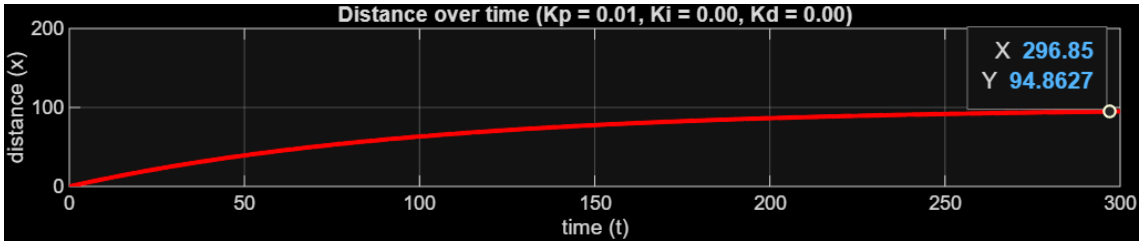


(c)

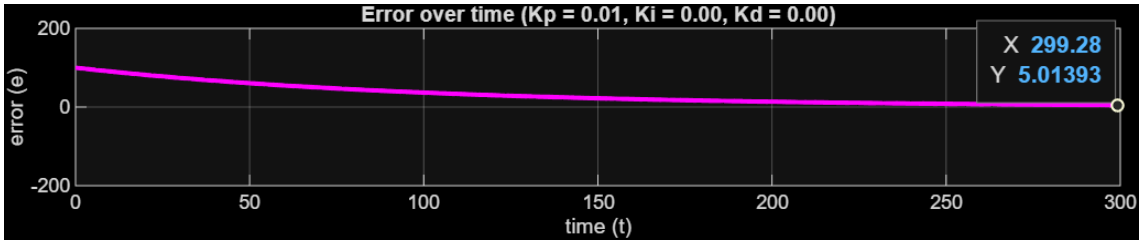
Figure 1. Proportional Loop output error and speed over time graph at Kp = 0.1: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)



(a)



(b)



(c)

Figure 1. Proportional Loop output error and speed over time graph at Kp = 0.01: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)

As we could see, the system responded directly towards the remaining error on an itteration. Kp Amplifies/dampen the amount of response proportionally as we could compare Fig 1 with 2. As we set Kp higher, the initial velocity immediately jumps to Kp times error (in an ideal system, in a physical system, there are consideration of the power safety during such extreme transient phase). For that reason also it has a steeper curve and a slower time to getting to the setpoint, or atleast trying to.

We consider here an ideal system where theres no physical imperfection, power complications, and outside interference. So when we try to design and tune an physical system, keep in mind there will be limitation on the coefficient we set. We could see that the function of error over time follows the same curve as the **diminishing exponent function**

$$error(t) = e^{-K_p t} \dots (1)$$

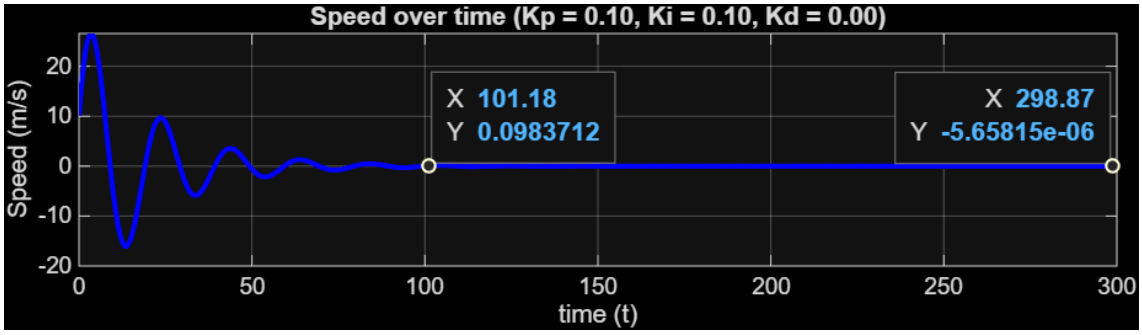
Where its asymtote is when error equals to 0. In other words, **the system does not actually reaches it set point, but only aproaching it**. This phenomenon could be seen on Fig 1.b where error does not reach 0 exactly. Considering the limitation of a physical system and enviromental disturbances, the **error gap overtime could be too great to just ignore**. This

error is called a **steady state error** for it happens if a system cannot reaches the satisfying set point even after it reaches its steady state phase. Systems like cruise control and even the goal system for boat suffers from this problem. Our boat **could possibly can't reach our altitude set point and if a slight disturbance happen, the boat cannot adjust them the same reason on how steady state error happen**.

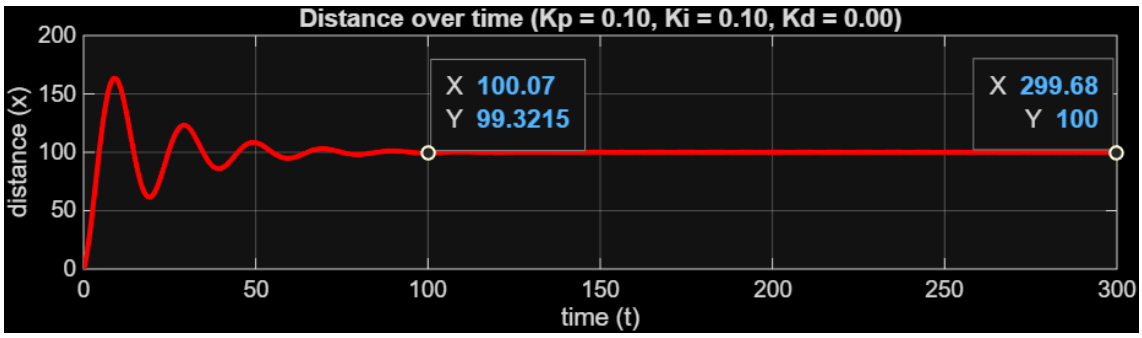
To caunterract this error, we calculate how much error **accumulated** over time, and then multiply them by some constant Ki for later tuning. Combining this, and we will get the PI Proportional and Integral (PI) loop.

$$K_p e(t) + K_i \int_0^t e(t)dt$$

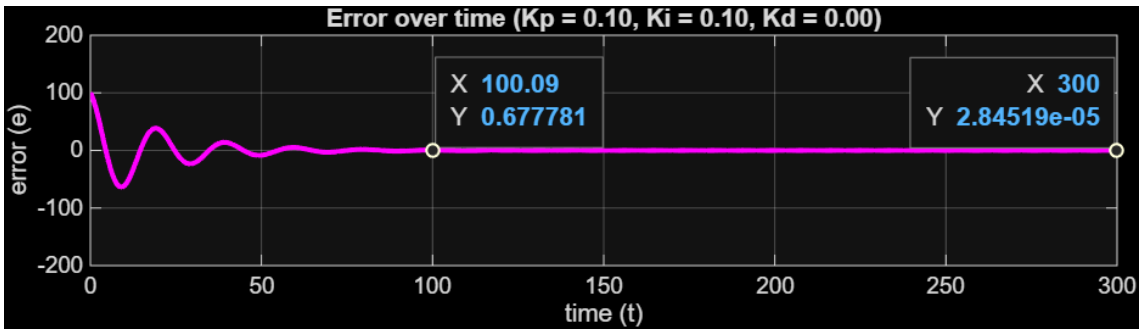
With this loop, we could accumulate enough error so that the system responded with the steady state error.



(a)

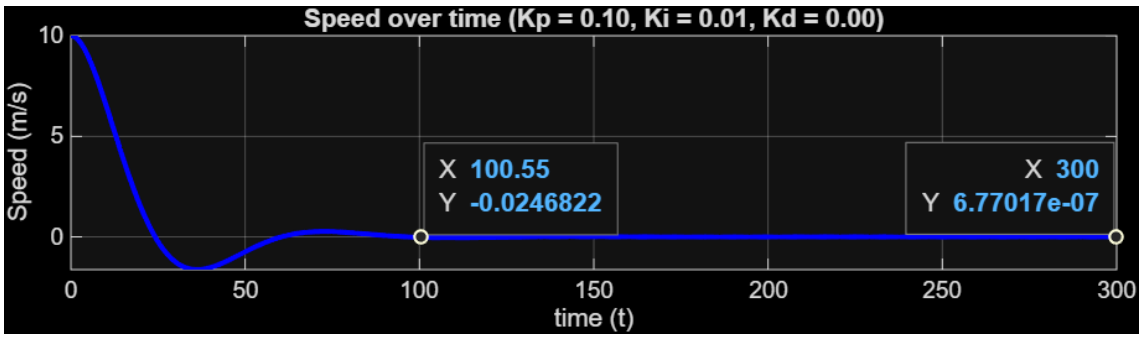


(b)

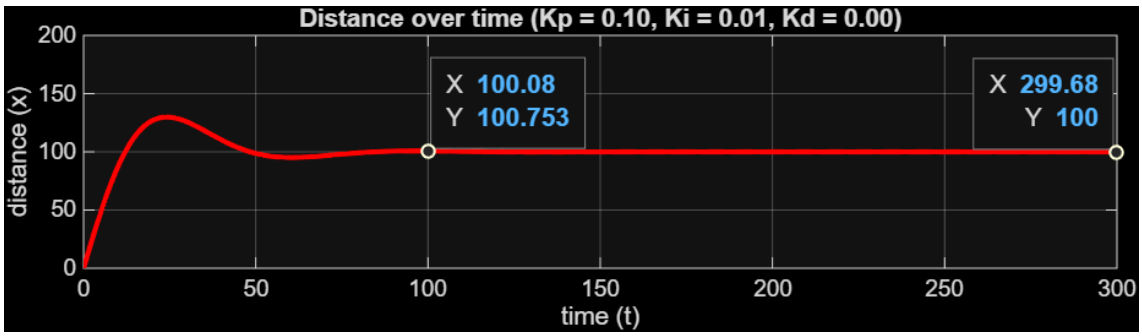


(c)

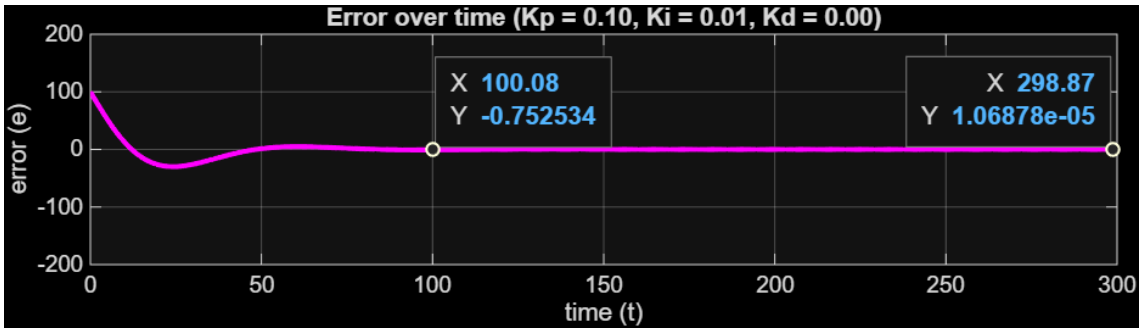
Figure 3. Proportional Loop output error and speed over time graph at Kp = 0.1 and Ki = 0.1: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)



(a)



(b)



(c)

Figure 4. Proportional Loop output error and speed over time graph at Kp = 0.01 and Ki = 0.01: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)

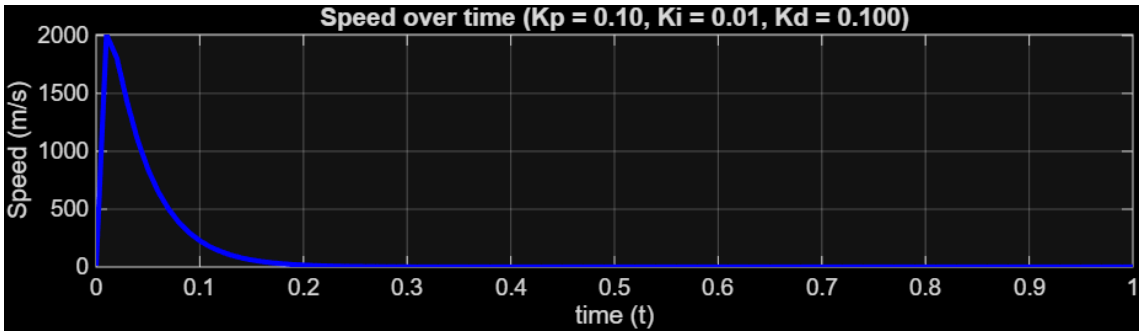
Shown in Fig. 3.a, we now know it is possible to achieve the set point of the system using PI loop as the integral loop keeps correcting itself as the error accumulates. But this design comes with another problem: at its transient phase, the system went so fast that it passes the set point and in Fig 3.a, it even passes 150 meters. This phenomenon is called an **overshoot**. But as it passes beyond the set point, both the P and I starts responding a feedback where the error is negative as it shown in Fig 3.c and Fig 4.c. which opposes the current

direction a tried to pull back to the set point. As this could happen more than one as in Fig 3.a, this loop creates a graph that resembles a **damped oscillation function**.

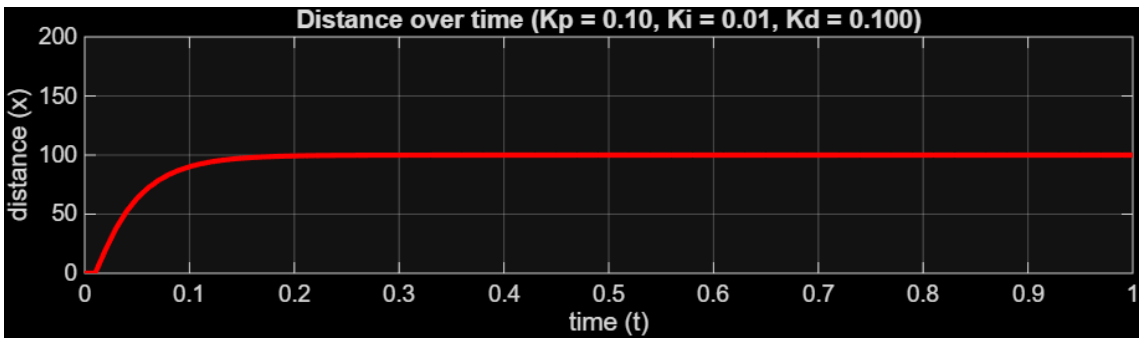
$$error(t) = e^{-\alpha t}\left(\text{Acos}(\omega_d t) + bsin(\omega_d t)\right) \quad ...(2)$$

In a physical system, the **overshoot could lead the boat to stall for its pitch goes beyond the safe number**.In a physical system also the **oscillation could keep going too long on the ships roll so that cruising becomes unstable and could also lead to stalling from trying to turn and if theres any disturbance on the water.** As we could see in Fig. 4, if we tune Kp and Ki well enough, we could get a fairly stable system, but we still can not afford an overshot above 25% on our heave, pitch, and roll on the boat since at that rate, stalling could happen and the **power management system could not operate under such extreme strain**. There needs to be an additional dampening system that could smooth out our response enough.

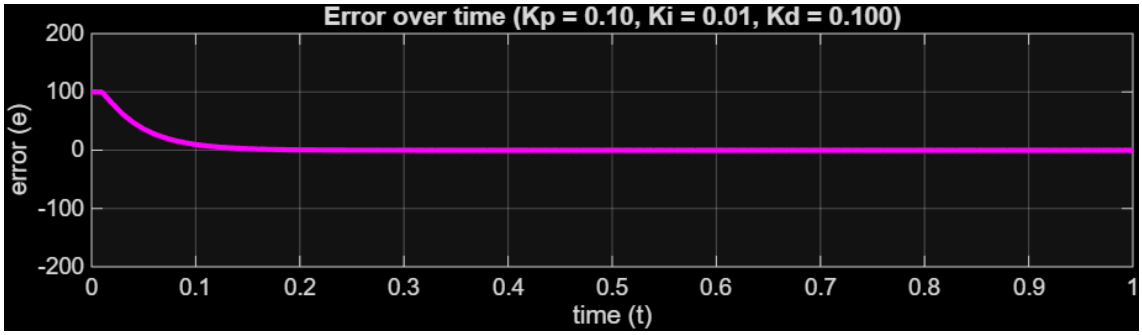
This is where differential comes to play. If we take the derivative/how much change the error over time, we could know how fast we're speeding and counterract it at hopes we could **dampen** the overshoot until its **critically damped**. We take that derivative and multiply it by a constant **Kd** so we could tune it. And finally, by adding Proportional, Integral, and Derivative, we get the full **PID loop**. The goal is now to tune Kp, Ki, and Kd until we get a critically damped response. From Fig. 4, our best result so far, we got to steady state at t around 100, around 35 meters overshoot, and a maximum speed of 10m/s.



(a)

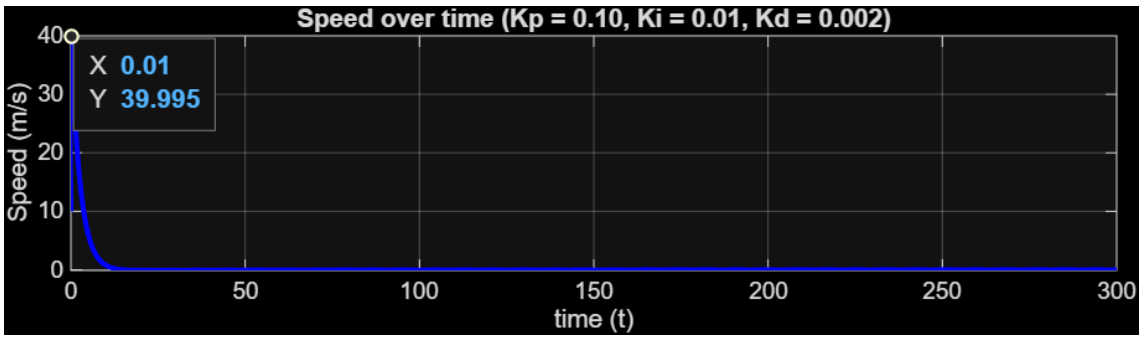


(b)

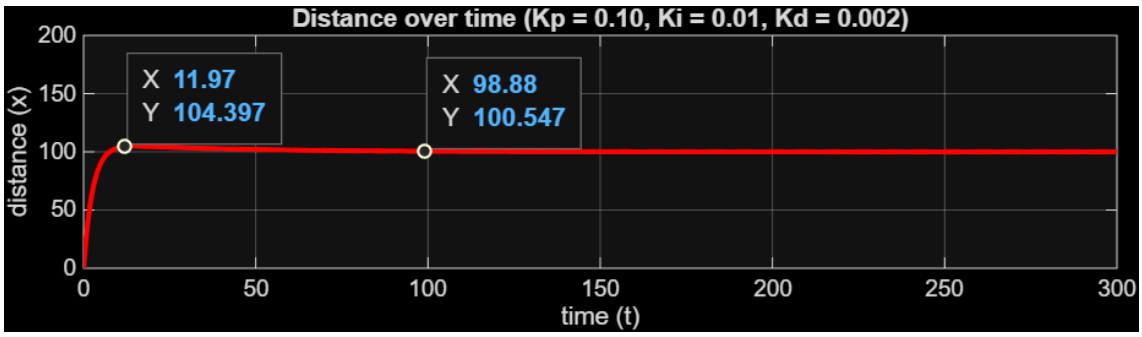


(c)

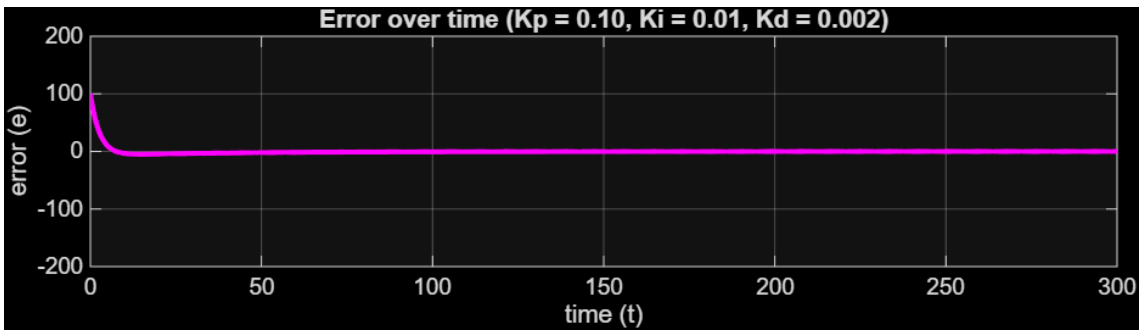
Figure 5. Proportional Loop output error and speed over time graph at Kp = 0.1, Ki = 0.1, Kd = 0.002: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)



(a)



(b)



(c)

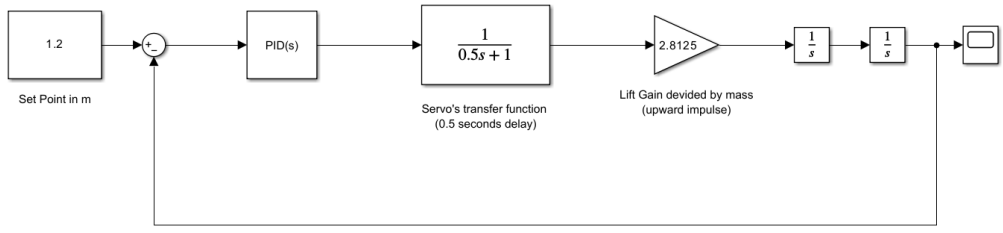
Figure 6. Proportional Loop output error and speed over time graph at Kp = 0.1, Ki = 0.1, Kd = 0.002: (a) Speed over Time (b) Distance over Time (c) Error over Time (Src. Personal Documentation)

From fig. 5, we've successfully damped the response and even lower the steady state to 1 seconds by Kd = 0.1. But, if we see carefully, we see that the initial speed jumps into a really high speed of 1000m/s. This is obviously not realistic, so we compromise our short steady state time to give our system a more realistic speed. That's what Kd = 0.002 on Fig. 6 tells us, which is good enough for our idealized PID simulation.

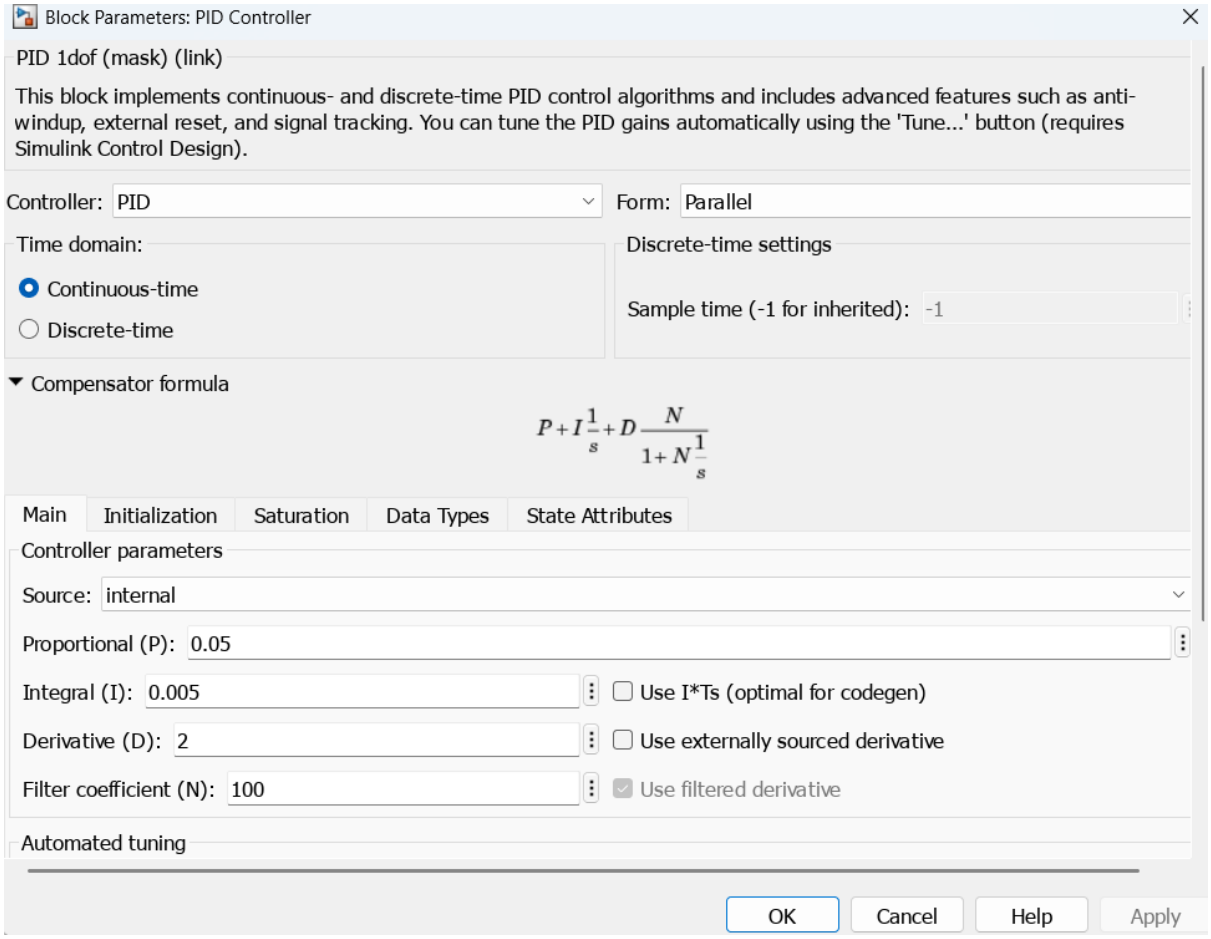
With PID loop, we could make transition from **hullborne to foilborn** automated and safe, creates a perfect damping to stabilize the ships orientation (roll and pitch) so that everything would be smooth sailing. **Inertial Mass Unit (IMU)** like MPU6050 Is used to for the roll and pitch sensor, while an **Ultrasonic sensor** like A02YYUW would fit to measure altitude. To calculate and to control the PID loop, a **microcontroller** like ESP32 or STM32 is used.

We will simulate our PID control using **simulink** to **simulate lifting up the hull after cruising at 15 m/s and maintaining a stable pitch**.

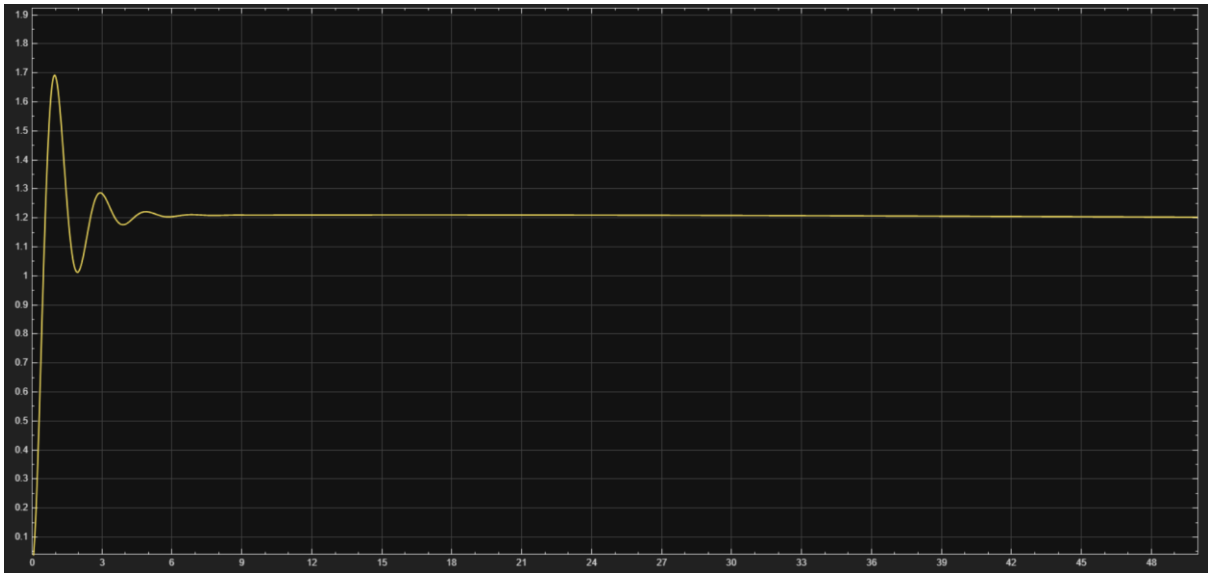
Using our ultrasonic sensor, we could measure the distance we have made for the hull to the water and in hullborne, we should be reading 0 meter, and on foilborne, it should be at 1.2 meters. So we set our set point to be 1.2, and we will feed it to our PID and our **servo plant**.



(a)



(b)



(c)

Figure 7. Altittude PID loop block diagram: a. Block Diagram, b. PID setting, c. Altitude output response (Src. Personal Documentation).

The gain 2.8125 was calculated by a set of formulas and assumtion as follows: We calculate dynamic pressure (q) by this formula:

$$q = \frac{1}{2}\rho v^2 \dots (3)$$

We assume the fresh water densitiy is 1000 kg/m^2, and we've decided that the minimum foilborne speed is 15 m/s. That gives us 112,500 N/m^2. Then we calculate the lift gain by this formula.

$$K_{lift} = qA\frac{dC_L}{d\alpha} \dots (4)$$

We assume the foil area A of 0.05 m^2 and the lift curve slope to be 0.1 per degree as we know below 15 degrees of angle attack, the lift grows proportionally with the angle. This gives us the lift gain of 562.5 N/degree. By the second newtons law,

$$F = K_{lift}\alpha = ma$$

$$\frac{a}{\alpha} = \frac{K_{lift}}{m} \dots (5)$$

Accelereation per degree of attack equals 562.5 N/degree devided by the vessel's mass of 200 kg assumed. And finally we get the gain of 2.8125 m/s^2 per degree to be plugged in after getting the servo's angle response. To achieve how much altitude we've gained, we just need to double integrate it to get displacement. As it is shown using double 1/s blocks. There we give feedback to loop back the system as it was shown on Fig. 7.a. We could also see on Fig. 7.b and c, we could improve the response by properly tuning PID, but for the sake of just block diagram demonstration, we wont need it.

Now lets get more complicated, what if we make a system that is controlled both the lift (heave) and pitch so that the boat would never flip over. To make the block diagram, we need to split the calculation of the **front foil** and **rear foil**. The front foil and rear foil will have a total surface area of 0.04 m^2 and 0.01 m^2 respectively. This is due to the **front foil being two-legged and the rear only one**. The dynamic pressure q is still the same since all foils moves at the the same speed relative with the waterflow. So considering these values and using Eq. 4, the amount of upward force per degree is 450 N/degree (F. Foil) and 112.5 N/deg (R. foil) and the gain we applied comes from those devided by the vessel's mass (200 kg). That gives us 2.25 front and 0.5625 rear gain. To calculate pitch gain, we need to convert transverse force into torque.

$$\tau = Fr$$

Torque is equal to the product of force and the its distance from the center of mass (CoM) r which we will assume its 0.5 meters front from the CoM (front) and 1.5 behind the CoM. With this, we got a torque value of 225 Nm (front) and 112.5 N (Rear). The moment of

inertia of the pitch movement of the boat we assumed to be 250 kgm^2. And by that, we could get the gain of 0.9 front and -0.675 back.

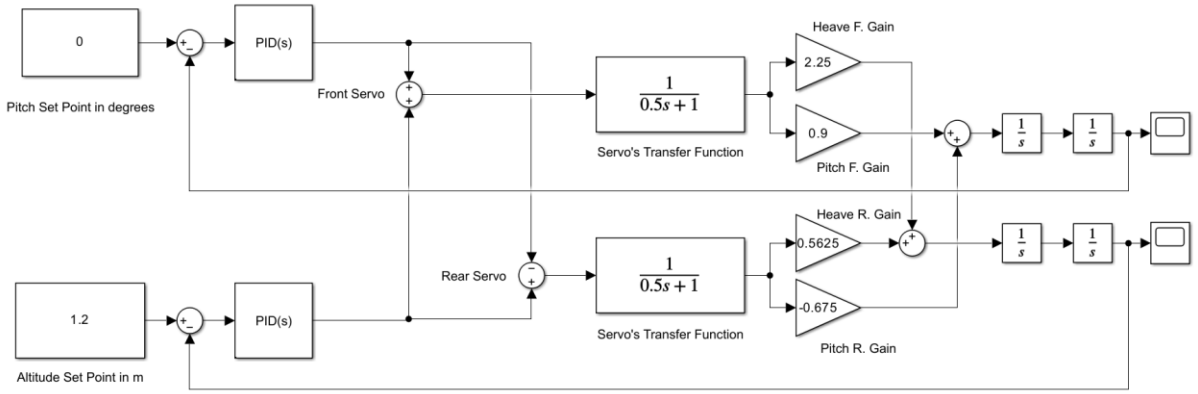


Figure 8. Heave and Pitch PID mix block diagram (Src. Personal Documentation)

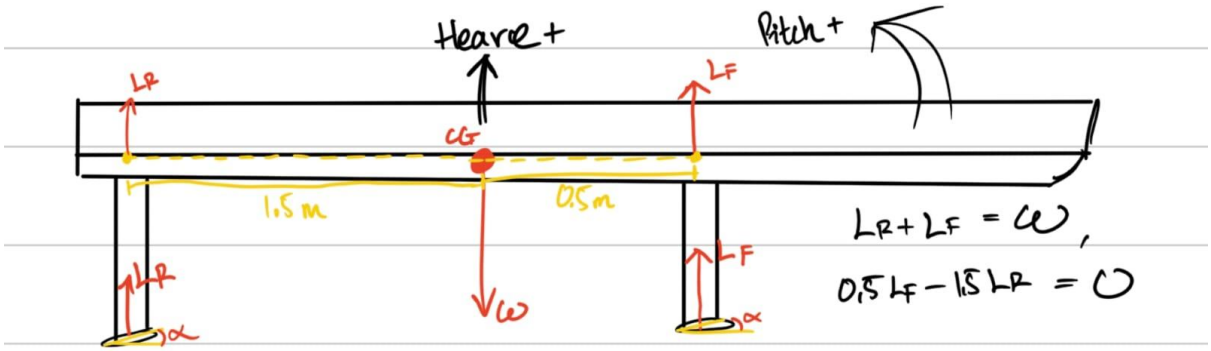


Figure 9. Vessel's force analysis (Src: document analysis)

Fig. 8 shows that both PID correction are split to front foil and and rear foil. In the front foil, we add both pitch and heave's correction. Meanwhile, on the rear part, we subract the heave correction with the pitch. This happens because if we set our pitch setpoint to be positive (vessel's nose going up), we weaken the back lift and reinforce the front one to create the maximum overall torque. This is why we need to declare that the back foil needs to be opposite to the front to achieve a certain pitch as shown in Fig. 9.

We use the same transfer function to describe the movement of the servo as it adjusts its angle, and then we plug ini each gain for each paralel lines. Before double-integrating it, we mix both pitch and both heave to get each output response.

Reference:

1. “Controling Self Driving Cars” by AerospaceControlsLab https://youtu.be/4Y7zG48uHRo?si=ITLe9BIskBGJrF5s
2. “What is a PID Controller?” by DigiKey https://youtu.be/tFVAaUcOm4I?si=nKnGXHVJ5cpw7pDQ
3. “PID Controllers in Simulink from Scratch for Beginners – Control Engineering Tutorials” by Haber https://youtu.be/U--pV35krq4?si=tG4P4WmUT_pgmwe1
4. “Hydrofoil design and optimization for a solar powered boat” by Tomaz, Carreira, and Amaro https://www.scribd.com/document/638660189/TSB-hydrofoil-analysis-and-optimization
5. “Developing Active Hydrofoil Stabilisation with Autonomous Steering” by Leincester
6. “Control Systems Engineering” by Nise
7. “Introduction to Flight” by Anderson